DS205.3 – Data Science with Python

Faculty of Computing

ASSESMENT GUIDELINES

Module Leader: Anton Jayakody

Overview

This document contains all the necessary information pertaining to the assessment of ***DS205.3 – Data Science with Python*****.** The module is assessed via **60% Examination and 40% Coursework**.

The sections that follow will detail the assessment tasks that are to be undertaken. The submission and expected feedback dates are presented in Table 1. All assessments are to be submitted electronically via the respective nLearn module pages before the stated deadlines.

	Submission Deadline	Feedback
Report + Coursework +Video	**TBD**	During Viva

Table 1: Assessment Deadlines

All assessments will be introduced in class to provide further clarity over what is expected and how you can access support and formative feedback prior to submission. Whilst the assessment information is provided at the start of the module, it is not necessarily expected you will start this immediately – as you will often not have sufficient understanding of the topic. The module leader will provide guidance in this respect.

Assessment 1: Coursework

Task:

This assignment contributes **40%** of the overall module mark for DS205.3 – Data Science with Python and is a **Group assignment**. Each Group should consist of maximum of **4 members**. The **final submission** must be submitted to nLearn by the specified submission date.

Problem Statement:

Despite the fluency of Large Language Models (LLMs), they suffer from two critical architectural flaws:

- training data cutoffs
- lack of access to private, specialized data.

Industry requirements have shifted toward Retrieval-Augmented Generation (RAG) and Multi-Agent Systems that utilize modular, extensible, and testable code.
Your task is to select a domain (e.g., Medical, Legal, Engineering, or Corporate Finance) and build a system that can accurately retrieve information from a private corpus and reason over it. Crucially, you must prove your system's accuracy through an empirical evaluation dataset.

Technical Requirements:

The system must be built using **Python 3.10+** and adhere to the following architectural standards:

1. **Modular OOP:** No monolithic scripts. Logic must be separated into packages.
2. **Data Engineering:** Find out the efficient/optimum way of handling PDFs for ingestion.
3. **Vector Persistence:** Use a persistent vector store to ensure data is not lost between sessions.
4. **Evaluation Framework:** Create a ground-truth dataset (Question/Answer pairs) and a script to programmatically calculate the faithfulness of your system's responses.

COURSEWORK DELIVERABLES

There are two main deliverables for this assignment. Only Deliverable D1, the final submission, and D2, the Demonstration and Viva, will contribute to the final module mark.

D1 – Final Technical Submission

This deliverable represents your primary engineering output. You are required to submit a comprehensive technical package to nLearn (max 50MB). Late submissions or broken code environments will result in significant point deductions.

1. Modular Python Source Code

Unlike previous modules, this submission must not be a single script. It must be a structured Python package consisting of multiple .py files that reflect professional architectural standards.

- ✓ Modular Architecture: *Your code must be organized into logical directories.*
- ✓ OOP Integrity: *Extensive use of Classes, and Dependency Injection is required.*
- ✓ Reproducibility: *You must include a requirements.txt file and a .env.example file. The Lecturer must be able to spin up your environment in a fresh Virtual Environment (venv) and run your system successfully*.
- ✓ Version Control (GitHub): *A link to a public GitHub repository is mandatory. Your commit history will be audited to verify the development lifecycle and individual contributions. Repositories with "single-push" histories (uploading all code at once) will be flagged for academic integrity review.*

2. Technical Design & Evaluation Report

A formal academic document (max 3,000 words) detailing the engineering decisions and the empirical performance of your system.

I. Problem Statement (ca. 500 words):

Identify the specific domain-specific knowledge gap your RAG/Agent system addresses. Why is a standard LLM insufficient for this task?

II. System Architecture & Design (ca. 600 words):

Explain your modular decomposition. Provide a diagram showing how data flows from a PDF to a Vector Store and finally to the LLM. Justify your choice of embedding models and chunking strategies (Size/Overlap).

III. Implementation & Traceability (ca. 600 words):

Detail the "Agentic Loop." Explain how you implemented "Traceability", show examples of what the system "retrieves" versus what it "generates."

IV. Empirical Evaluation (ca. 600 words):

You must provide results from your Evaluation Dataset. Present a table showing at least 10 "Ground Truth" questions, the system's responses, and an accuracy score (Manual or LLM-judged). Submissions without an evaluation dataset cannot achieve a Distinction grade.

V. Personal Reflection (ca. 400 words):

Reflect on the transition from procedural scripting to Object-Oriented AI development. What were the challenges in managing state within the Agentic loop?

VI. Future Work (ca. 300 words):

How would you scale this system? Discuss potential "Agentic" improvements, such as adding a "Reflection" step or Multimodal support for graphs/charts.

D2 – System Demonstration and Viva

This phase tests your ability to defend your architecture and prove that the AI's logic is a result of your engineering, not luck.

1. Video Demonstration (The "Trace")

Create an unlisted YouTube video (max 5 minutes). The video should not just show the final answer; it must show the overall process:
Ingestion: Show the code processing a PDF.
Retrieval: Show the system printing the "retrieved chunks" before it generates an answer.
Synthesis: Show the final grounded response.
Evaluation: Briefly show your evaluation script running.

2. The Viva (Oral Defense)

You will attend a live Viva with the module leader. You will be asked to:
Explain a specific design pattern used in your .py files.
Modify a small part of the prompt or logic on the fly to observe the change in system behavior.
Defend your evaluation metrics and explain any failures/hallucinations observed during testing.

Assessment Criteria:

Your work will be assessed according to the rubric found in Table 1. Your mark for this piece of coursework will be based on an aggregation of the marks for each category. Marks will be awarded based on **both the demonstration and the report**.

Category & Weighting	(< 40%)	(40-49%)	(50-64%)	(65-84%)	(85-100%)
Architectural Design (25%)	Monolithic or messy code. No use of OOP or classes.	Basic class structure used. Minimal separation of concerns.	Good use of OOP. Modular files. Some use of DI.	Professional use of ABCs and Interfaces. Highly decoupled logic.	Masterful design pattern application. Highly scalable and extensible.
Data & RAG Pipeline (30%)	Failed ingestion. High hallucinations. No vector DB.	Basic PDF extraction. Inefficient chunking (no overlap).	Working RAG pipeline. Correct vector storage and retrieval.	Advanced chunking logic. Clear evidence of context grounding.	Optimized retrieval. Handles complex PDF structures with high fidelity.
Evaluation & Bonus (15%)	No evaluation performed. Qualitative "guessing" only.	Informal testing with 1-2 examples.	Created a small test set. Basic manual comparison.	Ground-truth dataset used for objective scoring.	Automated scoring of faithfulness and relevancy.
Documentation & Git (20%)	No README. No commit history. Plagiarism suspected.	Basic README. Infrequent commits (all at once).	Clear instructions. Regular commits from all members.	Professional README. Clear technical diagrams. Detailed report.	Industry-standard documentation. Extensive commit history and CI/CD mindset.
Viva & Presentation (10%)	Unable to explain code logic. Video missing.	Basic explanation. Poor video quality.	Clear communication of system flow. Working video.	Expert defense of architecture. Clear demonstration of CoT.	Deep technical insight. Able to propose architectural optimizations.

Table 1: Feedback Template for Assessment 1

General Guidance

Plagiarism

All of your work must be your own. You must use references for your sources; however, you acquire them. Where you wish to use quotations, these must be a very minor part of your overall work.

To copy another person's work is viewed as plagiarism and is not allowed. Any issues of plagiarism and any form of academic dishonesty are treated very seriously. All your work must be your own and other sources must be identified as being theirs, not yours. The copying of another persons' work could result in a penalty being invoked.

Submission Guidelines

Ensure that you follow all submission guidelines provided for your coursework. This includes file formats, word limits, and any specific instructions related to the submission process. Late or improperly formatted submissions may result in penalties.

Penalties for Plagiarism

Understand the potential consequences of plagiarism and academic dishonesty. Penalties can range from receiving a failing grade on the assignment to more severe academic sanctions, such as suspension or expulsion. It is vital to approach your academic work with integrity and honesty.

Final Checks

Before submitting your work, perform a final check to ensure that all sources are properly cited, your quotations are correctly attributed, and your work adheres to the guidelines provided. Reviewing your work carefully can help avoid accidental plagiarism and ensure that your submission meets academic standards.

Referencing

Proper referencing is crucial for academic integrity and helps to avoid plagiarism. Use a consistent citation style throughout your work. Common citation styles include APA, MLA, Chicago, and Harvard. Ensure that every source you use is accurately cited in both in-text citations and the reference list.

Use of Quotations

Quotations should only be used to support your argument and must be properly attributed to their original source. They should not dominate your work. Aim to paraphrase and summarize information in your own words where possible and use direct quotations only when the original wording is essential for understanding or emphasis.